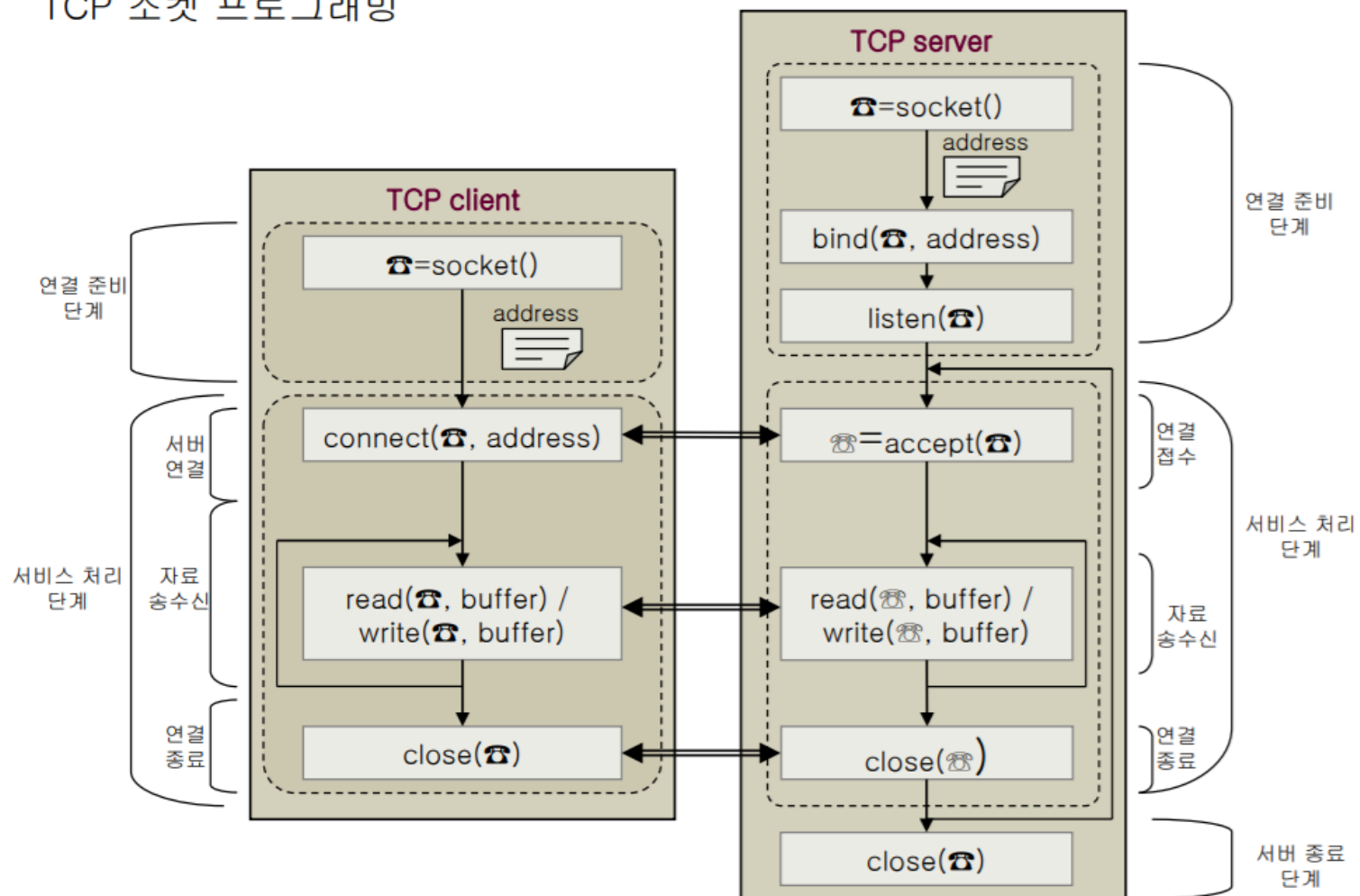


소켓 프로그래밍

7장 멀티프로세싱 방식의 다중 접속 서버

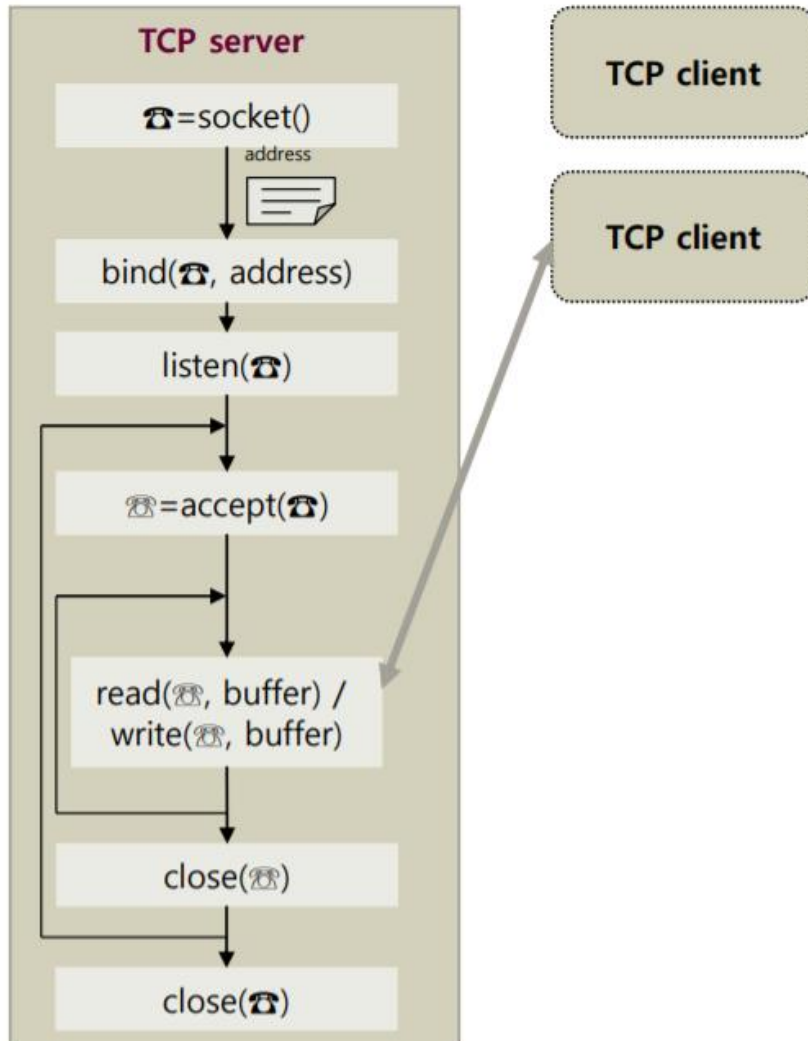
소켓 프로그래밍

TCP 소켓 프로그래밍



소켓 프로그래밍

TCP 서버 구조 (단일접속 서버)



```
main( ) {
    ....

    listen_s = socket(...);
    bind(listen_s, ...);
    listen(listen_s, ...);

    while(1) {
        conn_s = accept(listen_s, ...);
        do_service(conn_s);
        close(conn_s);
    }

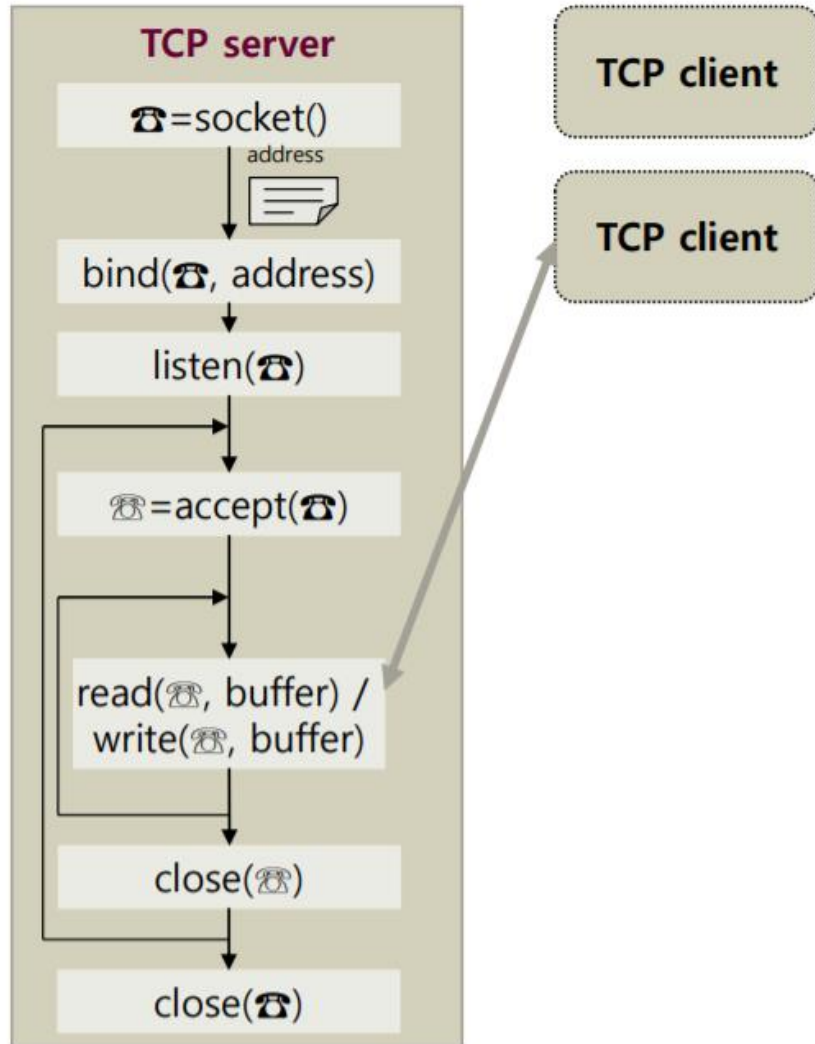
    close(listen_s);
}

do_service(int conn_s) {

    // 클라이언트 요청 처리
    read( ) / write( )
    ....
}
```

소켓 프로그래밍

TCP 서버 구조 (단일접속 서버)



한번에 한 클라이언트만 처리할 수 있는 구조

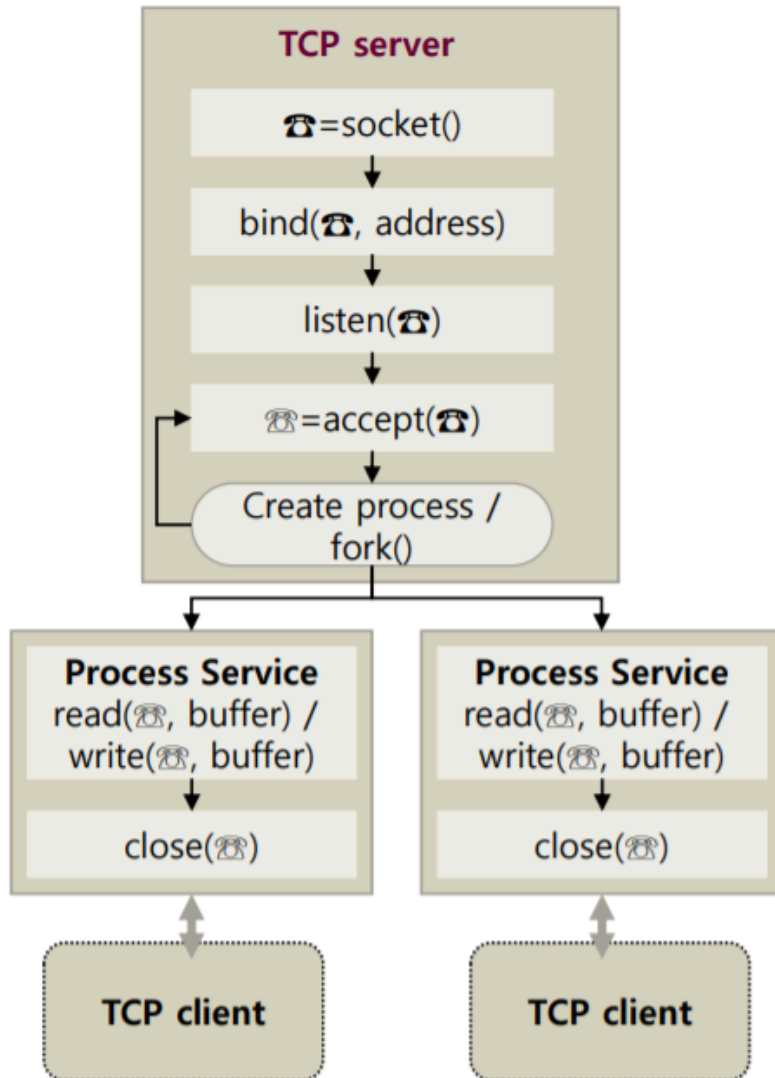


다중처리 구조

- 멀티프로세싱 (Multi-processing)
- 멀티쓰레딩 (Multi-threading)
- 멀티플렉싱 (Multiplexing)

소켓 프로그래밍

멀티프로세싱방식의 다중 접속 서버



```
main( ) {
    ....
    listen_s = socket(...);
    bind(listen_s, ...);
    listen(listen_s, ...);

    while(1) {
        conn_s = accept(listen_s, ...);

        if((pid = fork( )) > 0) { // 부모 프로세스
            // 다른 클라이언트의 요청 접수
            close(conn_s);
            continue;
        } else if(pid == 0) { // 자식 프로세스:
            // 클라이언트의 요청 처리
            close(listen_s);
            do_service(conn_s);
            exit(0);
        }
    }
}

do_service(int conn_s) {
    // 클라이언트의 요청 처리 루틴
    ....
}
```

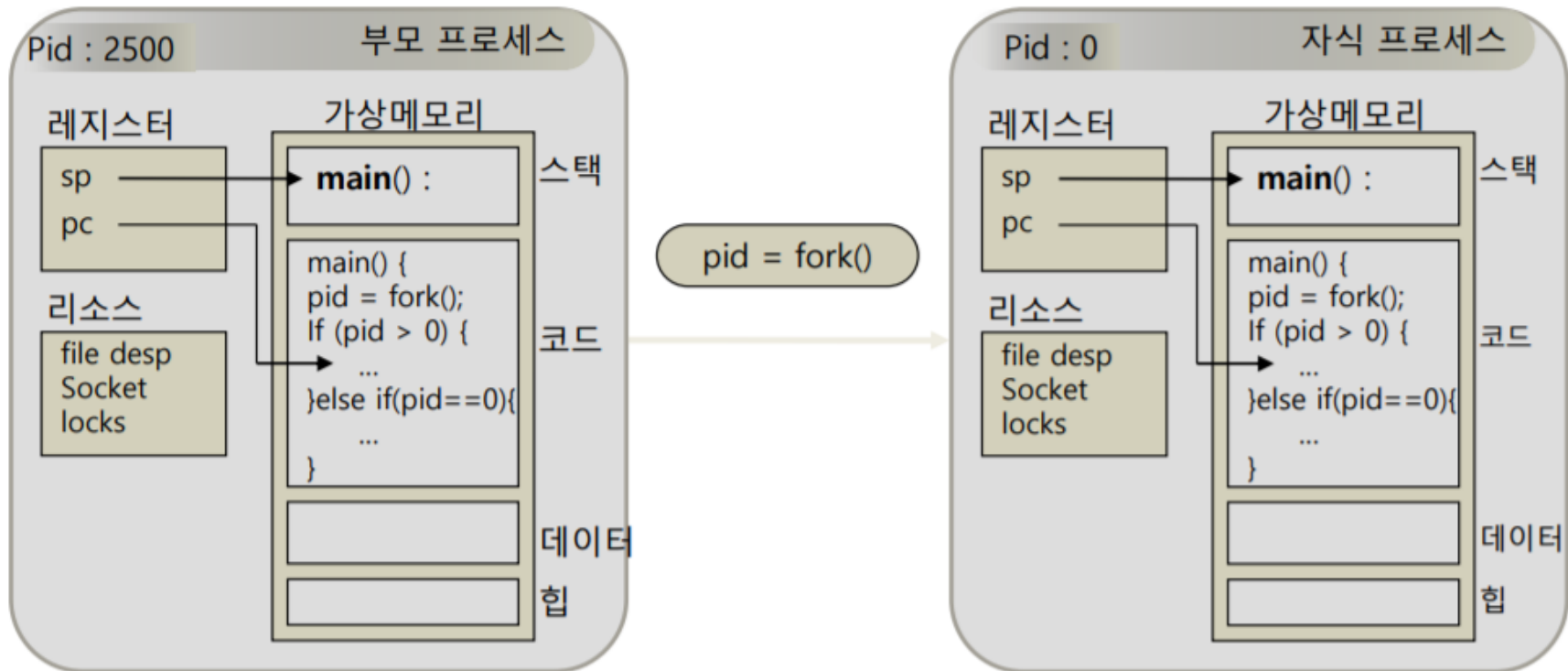
소켓 프로그래밍

fork 함수

```
fork

#include <sys/types.h>

int fork( );
```



소켓 프로그래밍

fork 함수

```
1 //fork_test.c
2 #include <stdio.h>
3
4 main( )
5 {
6     int    pid;
7     int    a = 10;
8     int    b = 20;
9
10    a = a + 1;
11
12    pid = fork( );
13    if(pid > 0) {
14        a = a + 1;
15        printf("PARENT");
16    } else if(pid == 0) {
17        b = b + 1;
18        printf("CHILD");
19    }
20    printf("[%d] a=%d/b=%d\n", getpid( ), a, b);
21 }
```

실행

실행 터미널-1

```
$ fork_test
CHILD[13786] a=11/b=21
PARENT[13785] a=12/b=20
$
```

소켓 프로그래밍

fork 함수를 이용한 다중 접속 서버의 구현

echo_server_fork.c

```
1 // echo_server_fork.c
2
3 ~~ 생략 ~~
4
5 9 #define PORT 8090
6
7 10
8 11 void do_echo(int);
9
10 12
11 13 main( )
12 {
13     ~~ 생략 ~~
14
15 18
16 19     pid_t pid;
17
18 20
19 21     listenSock = socket(PF_INET, SOCK_STREAM, 0);
20     ~~ 생략 ~~
21
22 27
23     if(bind(listenSock, (struct sockaddr *) &s_addr, sizeof(s_addr)) == -1) {
24         printf("Can not Bind\n");
25         return -1;
26     }
27
28 32
29     ~~ 생략 ~~
30
31 37
32     while(1) {
33
34 40         len = sizeof(c_addr);
35 41         connSock = accept(listenSock, (struct sockaddr *) &c_addr, &len);
36 42
```

echo 서버

```
43     if((pid = fork( )) < 0) {
44         printf("echo server can not fork( )\n");
45         return -1;
46     } else if(pid > 0) {
47         close(connSock);
48         continue;
49     } else if(pid == 0 ) {
50         close(listenSock);
51         do_echo(connSock);
52     }
53 }
54
55
56 void
57 do_echo(int connSock) {
58     int n;
59     char rcvBuffer[BUFSIZ];
60
61     while((n = read(connSock, rcvBuffer, sizeof(rcvBuffer))) != 0) {
62         printf("child\n");
63         write(connSock, rcvBuffer, n);
64     }
65     exit(0);
66 }
```

소켓 프로그래밍

```
pi@raspberrypi:~/network/ch7/t1 $ ./echo_server_fork &
[1] 2511
pi@raspberrypi:~/network/ch7/t1 $ ps
  PID TTY          TIME CMD
 1371 pts/0    00:00:00 bash
 2511 pts/0    00:00:00 echo_server_for
 2512 pts/0    00:00:00 ps
```

```
pi@raspberrypi:~/network/ch7/t1 $ ./echo_client
test1
original Data : test1
send (6)
recv (6)
echoed Data : test1
```

```
pi@raspberrypi:~/network/ch7/t1 $ ./echo_client
test2
original Data : test2
send (6)
recv (6)
echoed Data : test2
```

```
pi@raspberrypi:~/network/ch7/t1 $ ps
  PID TTY          TIME CMD
 1371 pts/0    00:00:00 bash
 2511 pts/0    00:00:00 echo_server_for
 2543 pts/0    00:00:00 echo_server_for
 2550 pts/0    00:00:00 echo_server_for
 2551 pts/0    00:00:00 ps
```

```
pi@raspberrypi:~/network/ch7/t1 $ ps
  PID TTY          TIME CMD
 1371 pts/0    00:00:00 bash
 2511 pts/0    00:00:00 echo_server_for
 2543 pts/0    00:00:00 echo_server_for <defunct>
 2550 pts/0    00:00:00 echo_server_for <defunct>
 2554 pts/0    00:00:00 ps
```

소켓 프로그래밍

```
#include <stdio.h>
#include <signal.h>

struct sigaction act_new;
struct sigaction act_old;

void sig_handler( int signo)
{
    printf( "First Ctrl-C pressed !! \n");
    sigaction( SIGINT, &act_old, NULL);
}

int main( void)
{
    act_new.sa_handler = sig_handler;
    sigemptyset( &act_new.sa_mask);
    act_new.sa_flags = 0;

    sigaction( SIGINT, &act_new, &act_old);
    while(1 )
    {
        printf( "sigaction test \n");
        sleep(1);
    }
}
```

```
pi@raspberrypi:~/network/ch7/t1 $ ./sigact
sigaction test
sigaction test
^CFirst Ctrl-C pressed !!
sigaction test
sigaction test
sigaction test
^C
pi@raspberrypi:~/network/ch7/t1 $
```

소켓 프로그래밍

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <sys/wait.h>

void do_child(void);
void do_parent(void);

void
sighandler(int n)
{
    int stat, chld_pid;
    sigset_t newset, oldset;

    chld_pid = wait(&stat);
    printf("signal ID    [%d]\n", n);
    printf("child pid    [%d]\n", chld_pid);

    if (WIFEXITED(stat)) {
        printf("normal  termination, exit status [%d]\n", WEXITSTATUS(stat));
    } else if (WIFSIGNALED(stat)) {
        printf("abnormal termination, signal number [%d]\n", WTERMSIG(stat));
    } else if (WIFSTOPPED(stat)) {
        printf("child stopped, signal number [%d]\n", WSTOPSIG(stat));
    }

    exit(1);
}
```

```
int main()
{
    int pid;
    struct sigaction sigset, oldset;

    sigset.sa_handler = sighandler;
    sigset.sa_flags = 0;

    sigemptyset(&sigset.sa_mask);

    sigaction(SIGCHLD, &sigset, &oldset);

    if ((pid = fork()) == 0 ) {
        do_child();
    } else if (pid > 0) {
        do_parent();
    }
}

void do_child()
{
    exit(100);
}

void do_parent()
{
    sleep(10);
}
```

소켓 프로그래밍

```
pi@raspberrypi:~/network/ch7/t1 $ gcc -o wait wait_sigchld.c
pi@raspberrypi:~/network/ch7/t1 $ ./wait
signal ID      [17]
child pid      [2654]
normal  termination, exit status [100]
```

Wait 함수로 자식 프로세스의 종료 상태 확인.

chld_pid = wait(&stat);

1. 자식 프로세스가 동작 중이면 호출 차단이 차단되기 때문에 상태를 얻어올 때까지 대기
2. wait() 함수 호출자가 시그널을 받을 때까지 대기
3. 자식 프로세스가 종료된 상태라면 즉시 호출이 반환되어 상태를 얻음, 이 때 wait() 함수는 자식 프로세스의 프로세스 ID를 반환

소켓 프로그래밍

```
// echo_server_fork_sig2.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <netinet/in.h>
#include <sys/socket.h>

#include <signal.h>
#include <sys/wait.h>
#include <errno.h>

#define PORT 8090

void do_echo(int);
void sigHandler(int);
```

```
main()
{
    int connSock, listenSock;
    struct sockaddr_in s_addr, c_addr;
    int len;

    pid_t pid;
    int state;
    struct sigaction act;

    act.sa_handler = sigHandler;
    sigemptyset(&act.sa_mask);
    act.sa_flags=0;

    state=sigaction(SIGCHLD, &act, 0);

    if (state != 0) {
        printf("signal error\n");
        exit(1);
    }
}
```

소켓 프로그래밍

```
listenSock = socket(PF_INET, SOCK_STREAM, 0);

memset( &s_addr, 0, sizeof(s_addr) );
s_addr.sin_addr.s_addr = htonl(INADDR_ANY);
s_addr.sin_family = AF_INET;
s_addr.sin_port = htons(PORT);

if (bind(listenSock, (struct sockaddr *)&s_addr, sizeof(s_addr)) == -1) {
    printf("Can not Bind\n");
    return -1;
}

if(listen(listenSock, 5) == -1) {
    printf("listen Fail\n");
    return -1;
}
```

```
while(1) {
    len = sizeof(c_addr);
    if((connSock=accept(listenSock, (struct sockaddr *)&c_addr, &len))<0) {
        if (errno == EINTR)
            continue;
        else
            printf("accept error\n");
    }
    if ( (pid = fork()) < 0 ) {
        printf("echo server can not fork()\n");
        return -1;
    } else if ( pid > 0 ) {
        close(connSock);
        continue;
    } else if ( pid == 0 ) {
        printf("child create\n");
        close(listenSock);
        do_echo(connSock);
    }
}
```

소켓 프로그래밍

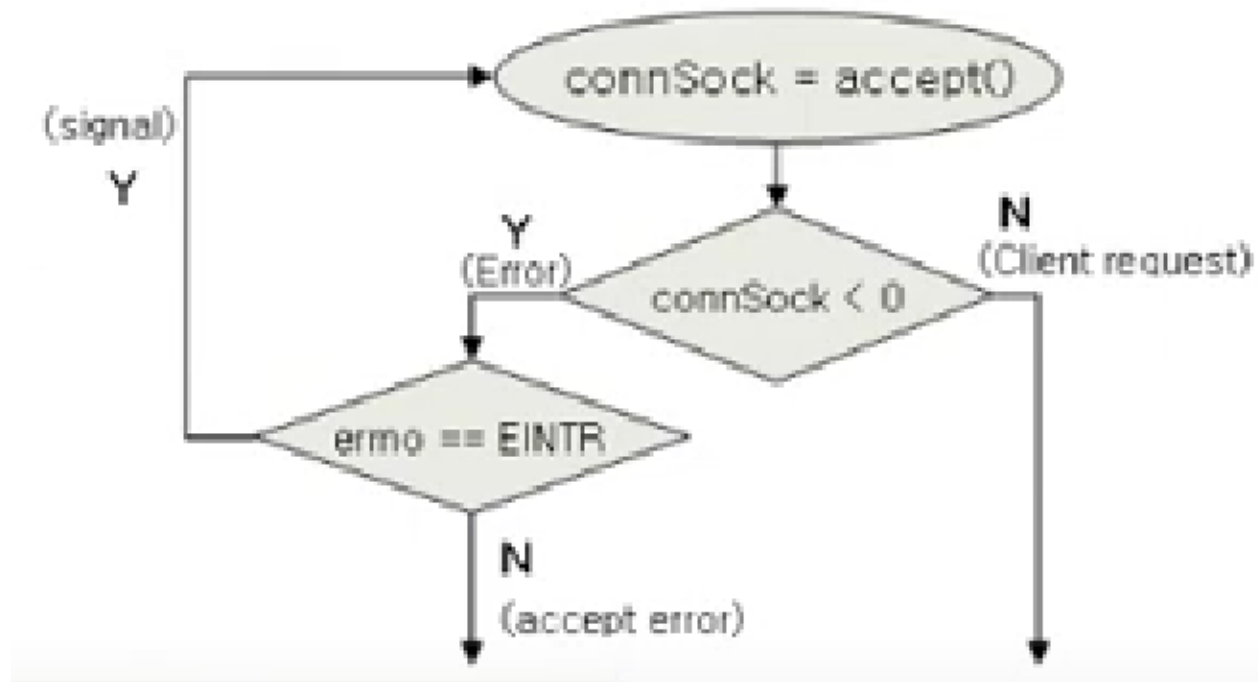
```
void do_echo(int connSock)
{
    int    n;
    char   rcvBuffer[BUFSIZ];

    while((n = read(connSock, rcvBuffer, sizeof(rcvBuffer))) != 0) {
        printf("child(%d)\n", n);
        write(connSock, rcvBuffer, n);
    }
    exit(1);
}

void sigHandler(int sig)
{
    int    pid;
    int    status;

    pid = wait(&status);
    printf("pid[%d] terminate\n", pid);
}
```

소켓 프로그래밍



소켓 프로그래밍

```
pi@raspberrypi:~/network/ch7/t1 $ ./echo_server_fork_sig2 &  
[1] 2733  
pi@raspberrypi:~/network/ch7/t1 $ child creat  
child(6)  
pid[2739] terminate
```

pi@raspberrypi: ~/network/ch7/t1

File Edit Tabs Help

```
pi@raspberrypi:~/network/ch7/t1 $ ./echo_client  
test1  
original Data : test1  
send (6)  
recv (6)  
echoed Data : test1  
quit
```

소켓 프로그래밍

10 주 과제

1. (멀티프로세스 관련) 서버를 `echo_server.c` 로 실행할때와 `echo_server_fork.c`로 사용할때의 차이점을 설명하시오.
2. (좀비프로세스 관련) `echo_server_fork.c` 에서 발생하는 문제점은 무엇인지 설명하고 그 문제점을 `echo_server_sig.c` 에서 어떻게 개선했는지 설명하시오.
3. (무한루프 관련) `echo_server_sig.c` 에서 발생한 문제점을 설명하고 이를 어떻게 개선했는지 설명하시오.
4. 완성된 `echo_server` 를 백그라운드로 실행하고 `echo_client` 를 2개 실행한다. 이 상황을 `ps` 명령어로 볼수 있도록 하고 화면 캡처한다(2개의 서버, 두개의 클라이언트)